Content- and Load-Aware Routing of Text and Multimodal Requests Across Heterogeneous CPU and Laptop Endpoints

CSE 291A Fall 2026 • Option 3: Adaptive Routing Across Heterogeneous Machines • Phase 1 Report

Jason Zhu, Fengfei Yu, Hanyu Li, Mengyao Ma, Zavier Zhao
{jaz019, fey006, hal209, mem032, zhz214}@ucsd.edu

1 Problem and Research Question

Cloud CPU instances and laptops differ in compute, memory bandwidth, capacity, and network distance. Qwen3.5 [2] (natively image–text-to-text; dense 0.8B–27B and MoE 35B-A3B) also offers tiers that trade quality for cost. Serving a request therefore couples three per-request decisions (*which tier*, *which machine*, and *when*) with a slower one: *which models* each machine keeps loaded. Phase 1 measures the differences, and Phase 2 derives the router from them.

Core question. *When should a text or image request run on each machine and model tier, given its content and shape, the fleet's load relative to capacity, and the cost of moving the request and its context?*

H1 *Shape dependence.* Prefill (compute-bound), decode (bandwidth-bound), and vision encoding scale differently across ISAs, core counts, and Apple-Silicon GPUs, so the best idle device depends on request shape.

H2 *Content predictability.* Cheap prompt features predict the smallest adequate tier and the output length well enough to pay for the decision.

H3 *Capacity regimes.* Below capacity, placement dominates. Above it, ordering, batching, and model residency matter more, and speed-oblivious policies misroute [3].

H4 *Communication.* Transfer is negligible for text inside AWS but not for images sent over the WAN to laptops.

2 Environment and Endpoints

Table 1: Endpoints. A–C and R share one AWS AZ; laptops are off-cloud. "Can host" lists memory-feasible Q4 models, loaded on demand; the laptop range depends on RAM (16 GB: up to 9B; 48 GB: up to 35B-A3B). Instance types are pending an eligibility check.

ID	Hardware	Can host
A	`c7g.xlarge`: Graviton3, 4 cores, 8 GB	0.8B–4B
B	`c7g.2xlarge`: Graviton3, 8 cores, 16 GB	0.8B–9B
C	`c7i.2xlarge`: Xeon SPR, 4C/8T, 16 GB, AMX	0.8B–9B
L	5 MacBook Pro/Air: Apple Silicon (Metal), 16–48 GB	0.8B–35B-A3B
R	`c7g.medium`: router + load generator	–

Core and extended study. The core study follows the option specification: three AWS CPU instance types and the router in one Region, over private IPs. Each pair changes one factor. A vs. B varies cores and memory on one ISA. B vs. C varies the ISA (Arm Neoverse-V1 vs. x86 with AVX-512/AMX) at equal vCPUs and memory; C has half of B's physical cores because an x86 vCPU is a hyperthread. The extended study adds five Apple-Silicon MacBooks (Metal GPU, unified memory), reached over the WAN through a WireGuard tunnel, which is a second communication regime.

Runtime and models. Every device runs one pinned llama.cpp release [1] (`llama-server`; images via `mmproj`), built natively per backend (Arm, x86, Metal), with identical SHA-256-verified GGUF files and thinking mode off. Devices are *not* bound to one model: every GGUF that fits is pre-staged on disk, and a per-device agent starts or stops one `llama-server` per resident model at the router's request.

Confounder control. Hardware comparisons use matched models (0.8B and 2B at Q4_K_M on every device); each device's best practical configuration is reported separately. Results that change the tier are always reported with answer quality, so gains from a smaller model are never credited to routing.

3 Workloads and Metrics

Requests. Text: **T1** short chat; **T2** prefill-heavy (2–8K-token document, short answer); **T3** decode-heavy; **T4** multi-step agent sessions with accumulating context and tool delays. Image: **M1** single image at three resolutions; **M2** chart or document; **M3** multi-image. Prompts come from public chat traces and from benchmarks with programmatic checks (GSM8K, MMLU; ChartQA, TextVQA), whose varying difficulty yields tier labels (E5).

Load. Open-loop Poisson and on/off bursty arrivals are replayed from fixed seeds at offered loads $\rho = 0.5$–1.5 of measured fleet capacity, covering both regimes. Policies are tuned on dev mixes and evaluated on held-out mixes with different text:image ratios and lengths. Performance runs fix output length (`n_predict`, `ignore_eos`); quality runs terminate naturally.

Metrics. Per request: TTFT, TPOT, end-to-end latency, queueing delay, chosen (device, tier), correctness, and status (timeouts and rejections are kept). Per device at 1 Hz: CPU/GPU utilization, memory, slot occupancy, and resident models. Aggregates: throughput, p50/p99 TTFT and TPOT, SLO attainment, accuracy vs. always-largest, network bytes, decision latency, and swap time. Versions are pinned, and runs are repeated at least three times (median, bootstrap 95% CI). Cold and warm start are reported separately, and laptops run on AC power in a fixed performance mode.

4 Phase 1 Measurements

E1 *Isolated profiles:* device × model × quant × lengths × image resolution → prefill rate, TPOT vs. context, vision-encode time, and memory.

E2 *Load curves:* slots 1–8 and arrival rate up to saturation → latency vs. load, capacity, and batching gain.

E3 *Communication and state:* RTT and bandwidth (intra-AZ vs. WAN), transfer time vs. payload, warm vs. switched sessions, and router overhead.

E4 *Model lifecycle:* load and unload time per (device, model), and co-location interference.

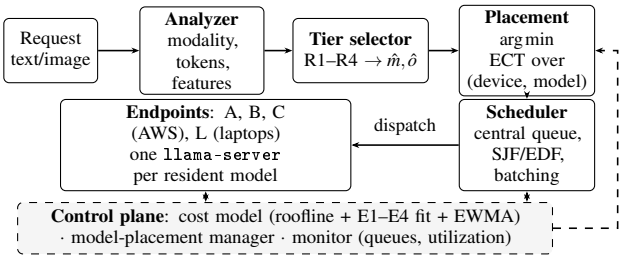


Figure 1: Router architecture. Solid arrows: per-request data plane. Dashed: control plane (model load/unload, metrics, queue state, cost updates). The placement manager changes which models are resident on each device.

E5 *Tier labels:* each dev prompt on each tier → the smallest passing tier and the output length.

E1–E4 calibrate the cost model and the crossover maps (the best device per class, idle vs. loaded); E5 trains the tier selector.

5 Phase 2: Router Architecture

Expected performance from model configuration. The router (Fig. 1) needs a cost model. For model m on device i, a roofline estimate gives prefill time and decode time per token:

$$P_{i,m}(p) \approx 2N_m p \,/\, (\eta_i^c F_i), \tag{1}$$

$$\mathrm{TPOT}_{i,m}(b) \approx \max\{(S_m + bK_m)/(\eta_i^m \beta_i),\ 2bN_m/(\eta_i^c F_i)\}. \tag{2}$$

p is the number of prompt tokens and b the decode batch size. N_m is the number of active parameters (3B for 35B-A3B), S_m the quantized weight bytes, and K_m the per-sequence KV/state bytes read per step. F_i and β_i are peak FLOP/s and memory bandwidth. The efficiencies η are fitted from E1–E2 and EWMA-corrected online, and vision encoding $V_i(v)$ is fitted in image tokens v. The decode term shows that batching is nearly free until the device becomes compute-bound. The estimated completion time of request r on (i,m) is

$$\begin{aligned}\mathrm{ECT}_{i,m}(r) = W_i + V_i(v_r) + P_{i,m}(p_r) + \hat{o}_r\,\mathrm{TPOT}_{i,m}(b_i) \\ + \mathrm{RTT}_i + s_r/\mathrm{BW}_i + C_i(r) + L_{i,m},\end{aligned} \tag{3}$$

where W_i is the work queued on i, $\hat{o}_r$ the predicted output length, and s_r the payload size. C_i is the re-prefill penalty when the session's state is elsewhere, and $L_{i,m}$ is the load time when m is not resident.

Content-based tier selection (the core decision). The selector predicts the smallest adequate tier $\hat{m}(r)$ and an output-length bucket $\hat{o}_r$, using mechanisms of increasing cost:

R1 *Rules:* modality, length thresholds, and regex cues (code, math, "prove"/"explain").

R2 *Classic NLP:* TF-IDF and hand-crafted features (length, math/code density, question type) fed to logistic regression or boosted trees; sub-millisecond on a CPU.

R3 *Tiny-LM router:* Qwen3.5-0.8B scoring a one-token label, or a small fine-tuned encoder (cf. RouteLLM [4]). More accurate, but its prefill is charged to TTFT.

R4 *Cascade:* R2 decides, and R3 is called only when R2's confidence is below θ (cf. FrugalGPT [5]).

All four are trained on E5 labels and judged on tier accuracy, end-task quality, and decision latency relative to service time.

Scheduling below and above capacity. Below capacity, each request goes greedily to $\arg\min_{(i,m)}$ ECT over feasible pairs (tier at least $\hat{m}$, fits in memory), so a slow idle device is used only if it finishes sooner. Above capacity or during bursts, requests wait in a central router queue and are bound to an endpoint only when a slot frees (late binding). The queue is ordered by predicted size with aging (SJF) or by deadline (EDF). Small same-tier requests are batched, a request may be downgraded within an explicit quality budget, and admission control sheds load; rejections count as failures.

Dynamic model placement. Every Δ seconds the placement manager re-plans which models are resident on each device from per-tier demand forecasts (EWMA arrival rates), within memory limits. For example, it can start an extra 0.8B server on B or a laptop when small requests surge, so that they can be batched. It changes residency only when the expected gain over the planning horizon exceeds the measured load time and co-location interference (E4), with hysteresis against thrashing.

Baselines and evaluation. Baselines: always-i, round-robin, least-outstanding-requests, weighted round-robin tuned on dev, static device-to-model binding, and always-largest (the quality reference). We evaluate steady and bursty arrivals across ρ on held-out mixes, with the fastest device idle and busy. We ablate R1–R4, analytic vs. measured cost, oracle vs. predicted tier and length, the scheduling policy, and dynamic vs. static placement. A simple policy sufficing is a valid outcome.

Scope and risks. Must-have: the cost model, ECT placement, R1–R2, both load regimes, and all baselines. Stretch: R3–R4 and dynamic placement. Risks: (1) Qwen3.5 vision in llama.cpp is not yet verified on every backend. (2) Laptop/WAN variability (repeated runs, reported variance). (3) Budget: the team's $5 \times \$25 = \$125$ buys about 150 h of A+B+C+R at about \$0.83/h (to be confirmed); we develop on laptops, auto-shut down instances, and set billing alarms.

References

[1] llama.cpp. `https://github.com/ggml-org/llama.cpp`

[2] Qwen Team. Qwen3.5 model collection. `https://huggingface.co/collections/Qwen/qwen35`

[3] M. Harchol-Balter. *Performance Modeling and Design of Computer Systems*. Cambridge Univ. Press, 2013.

[4] I. Ong et al. RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665, 2024.

[5] L. Chen, M. Zaharia, J. Zou. FrugalGPT. arXiv:2305.05176, 2023.

Appendix A: Timeline and Milestones

Stage	Work and milestones (Thanksgiving, Nov 26–29, is buffer)
1. Environment setup Nov 3–8	Confirm eligibility and quota; provisioning scripts; pinned llama.cpp builds; GGUF/`mmproj` on every device; WAN tunnel; model agent; load generator and logging; pass-through router. **M0 (Nov 4):** Phase 1 report. **M1 (Nov 8):** every device serves 0.8B text and image requests through the router.
2. Config. testing & expectation setup Nov 6–18	E1–E5 sweeps (models, quantizations, lengths, image resolutions, load, network, model lifecycle); fit roofline efficiencies; collect tier labels; freeze dev/held-out traces. **M2 (Nov 18):** calibrated cost model, labeled dev set.
3. Router development Nov 16–28	Baselines; ECT placement; R1–R2, then R3–R4; central-queue scheduler; placement manager (stretch); decision-latency microbenchmarks. **M3 (Nov 25):** router and baselines run end-to-end on dev mixes.
4. Experiments & analysis Nov 25–Dec 10	Held-out runs across ρ (steady and bursty; fastest device idle and busy); ablations; repeated runs with CIs; failure analysis; report and artifacts. **M4 (Dec 5):** all runs complete. **M5 (Dec 10):** final report.